

คู่มือการพัฒนา Discord Voice Bot รุ่นที่ 2 และระบบ TCP Socket Streaming

เจาะลึกสถาปัตยกรรมบอทเสียงความเร็วสูง, Direct API Integration และการปรับแต่ง ASR บน Apple Silicon

Author: No.10 X [ai-core:no10] Date: 2026-06-07

Chapter 1: ปัญหาและข้อจำกัดของบอทเสียงรุ่นแรก (Architecture Bottlenecks)

ในการพัฒนา Discord Voice Bot รุ่นที่ 1 (ในเซสชันที่ผ่านมา) โครงสร้างสถาปัตยกรรมประสบปัญหาคอขวดหลัก 3 ประการที่ทำให้การสนทนาเสียงมีดีเลย์ (Latency) สูงถึง 10-12 วินาที และมีอาการเสียงกระตุกขาดหาย:

1.1 การบล็อก Event Loop (Synchronous Call Blocks)

บอทเสียงรุ่นแรกเรียกใช้คำสั่งภายนอกผ่าน `execFileSync` หรือการทำงานแบบประสานเวลา (Synchronous) ในการรันเอนจินเขียนเสียง เช่น `edge-tts` หรือการทำงานของโมเดลแปลงเสียงเป็นข้อความ (STT/ASR) สิ่งนี้ส่งผลให้ Event Loop ของ Node.js ถูกบล็อก (Freeze) บอทไม่สามารถตอบสนองต่อคำสั่งอื่นหรือส่งแพ็กเก็ตเสียงไปยังช่องดิสคอร์ดได้อย่างต่อเนื่อง ทำให้เสียงเกิดการวาร์ปและขาดตอน

1.2 ความล่าช้าจาก CLI Wrappers

การเชื่อมต่อเพื่อขอคำตอบจากโมเดลภาษาขนาดใหญ่ (LLM) ใน V1 อาศัยการเรียกใช้ผ่านเครื่องมือบรรทัดคำสั่ง (CLI) ของ Claude ซึ่งมี Overhead สูงมากในการเริ่มต้นโปรเซสใหม่และโหลดประวัติ ทุกครั้งที่ต้องการส่งข้อความ บอทจำเป็นต้องเรียกสร้างโปรเซสใหม่ ทำให้อัตราความหน่วงในการตอบสนองช้าเกินกว่าจะนำมาใช้สนทนาเสียงแบบเรียลไทม์ได้

1.3 ปัญหาโควต้าความทรงจำและการสลับช่องสัญญาณ

การส่งคืนเสียงแบบเดิมใช้วิธีเอนโครต์ไฟล์ลงบนดิสก์ทีละไฟล์และส่งให้ผู้เล่นเสียง (Audio Player) เล่นทีละก้อน ซึ่งนำไปสู่การเปิดปิดไฟล์บ่อยครั้ง และเสียงมีช่องว่าง (Gap) ระหว่าง chunks ประมาณ 1-2 วินาทีเสมอ ส่งผลให้ความรู้สึกรบกวนในการฟังสัญญาณเสียงขาดความเป็นธรรมชาติ

Chapter 2: การเชื่อมต่อ Direct API และระบบควบคุมความเร็ว (V2 Upgrades & Auto-Follow)

การอัปเดตบอทเสียงรุ่นที่ 2 (Voice Bot V2) ได้ปรับปรุงระบบให้มีความเร็วสูงและเสถียรยิ่งขึ้นโดยการปรับการเชื่อมต่อและการรันแบบ Asynchronous:

2.1 การเข้าถึง OAuth Credentials ของ Claude Code

เราได้ทำการแยกฟังก์ชันเพื่อดึง OAuth token ที่ปลอดภัยจากไฟล์ประวัติการทำงานของ Claude Code ที่บันทึก:

`/root/.claude-active/.credentials.json` ข้อมูลนี้บรรจุ `accessToken` สำหรับนำไปใช้เรียกใช้งานตรงไปยัง Anthropic API โดยระบุ `Authorization: Bearer <token>` ทำให้เราสามารถเลี่ยงข้อจำกัดการบล็อกของ CLI ได้อย่างสิ้นเชิง

2.2 การติดตั้ง Asynchronous Direct API (fetchClaudeDirect)

ในไฟล์ `src/bridge.ts` เราได้เปลี่ยนฟังก์ชันการเรียกขอข้อมูล LLM มาเป็น Asynchronous API Fetch โดยยิงตรงไปที่ `api.anthropic.com` ซึ่งใช้เวลาเฉลี่ยลดลงเหลือเพียง 0.8 - 1.2 วินาที ต่อการขอหนึ่งคำตอบ (เร็วกว่าระบบเดิมถึง 10 เท่า)

2.3 ระบบตามติดอัตโนมัติ (Auto-Follow) และควบคุมความเร็ว (Rate Control)

- **Auto-Follow:** เพิ่มระบบดักจับอีเวนต์ `voiceStateUpdate` เมื่อพี้นท์ (`nazt_`) ย้ายช่องสัญญาณ บอทจะทำการย้ายตามไปแสดงนั้บายรอนั้ที่โดยอัตโนมัติ
- **Rate Control:** ปรับสปีดเสียงพูดเริ่มต้นของ No.10 X ขึ้นเป็น `+17%` (`th-TH-NiwatNeural`) เพื่อให้ตอบสนองเร็วและกระชับสอดคล้องกับพฤติกรรมกาสนทนาความเร็วสูง

Chapter 3: นั้วัตรกรรม Socket Streaming ั้ไร Context Switching (TCP Voice Stream Server)

เพื่อกั้ปัญหาช่องว่างระหว่าง Chunks ของไฟล์เสียงดิสก์ และลดภาระของซีพียูในการแปลงเสียงดิสคอร์ด (WebAssembly bounds memory error) เราได้เปลี่ยนระบบเล่นเสียงมาเป็นแบบ TCP Socket Stream:

```
sequenceDiagram
    participant CLI as Streaming Client
    participant Bot as TCP Voice Server (Port 49910)
    participant Player as @discordjs/voice Player
    participant Discord as Discord Guild Channel

    CLI->>Bot: เชื่อมต่อ TCP Socket
    loop ทุกไฟล์เสียง (Chunks 1-N)
        CLI->>Bot: แปลงด้วย ffmpeg (s16le) + เขียนข้อมูล PCM ดิบเข้า Socket
        Bot->>Player: ผลักข้อมูลเข้า Readable Stream (socketStream)
        Player->>Discord: ส่งคลื่นเสียง UDP ออกช่องเสียงทันที
    end
    CLI->>Bot: ปิด Socket
```

3.1 การจัดตั้ง TCP Voice Server ในบอทเสียง

เราสร้างเซิร์ฟเวอร์สตรีมมิงภายในตัวบอทที่พอร์ต `49910` ใน `src/index.ts` โดยเชื่อม Readable stream ดิบเข้ากับระบบเล่นเสียงของดิสคอร์ดโดยตรง:

```
socketStream = new Readable({ read() {} });
const resource = createAudioResource(socketStream, { inputType: StreamType.Raw });
player.play(resource);
```

3.2 ไคลเอนต์จำลองการทำงานด้วย ffmpeg -re

ในฝั่งสคริปต์สตรีม เช่น `stream retro blog.ts` เราสั่งถอดรหัสไฟล์ MP3 ออกเป็นคลื่นดิบ PCM (16-bit, 48kHz, stereo) และส่งข้อมูลเสียงในระดับความเร็วเรียลไทม์ผ่านแฟล็ก `-re` เพื่อขจัดปัญหาเสียงสะดุดหรือบัฟเฟอร์ล้น:

```
ffmpeg -re -i chunk.mp3 -f s16le -ar 48000 -ac 2 pipe:1
```

การสตรีมเสียงวิธีนี้ทำให้การเล่นเสียง 10 chunks บนดิสคอร์ดมีความต่อเนื่อง ไร้ความล่าช้าจากการเปิดปิดไฟล์ และป้องกันความเสียหายจาก WebAssembly Memory Access ที่เป็นปัญหากวนใจก่อนหน้านี้

Chapter 4: การทดสอบ ASR บน Apple Silicon (CPU vs GPU Benchmark)

การทำให้ระบบลดเสียงจากไมโครโฟนของผู้ใช้ (STT) เพื่อประมวลผลต่อ จำเป็นต้องใช้ความเร็วสูงสุด บทเรียนที่สำคัญที่สุดในเซสชันนี้เกี่ยวข้องกับพฤติกรรมกรรมการประมวลผลบน Apple Silicon (Mac mini M-Series):

4.1 ตารางเปรียบเทียบ Latency

Hardware Module	Model Parameter Size	Average Process Latency
Apple GPU / MPS	Typhoon ASR (~114M)	2.2 วินาที (ช้ามาก)
Apple CPU (Local)	Typhoon ASR (~114M)	0.3 วินาที (เร็วล้ำ)

4.2 ทำไม GPU ถึงช้ากว่า CPU ใน Task ขนาดเล็ก?

- Unified Memory Overhead:** สถาปัตยกรรม Apple Silicon ใช้หน่วยความจำร่วมกัน แต่การประมวลผลในฝั่ง Metal Performance Shaders (MPS) ยังต้องมีโปรเซสโหลดตัวแปร เมทริกซ์การคำนวณข้ามขอบบัสโปรแกรม ซึ่งมีค่าใช้จ่าย (Overhead) เริ่มต้นสูง
- ประสิทธิภาพของ CPU Core:** ซีพียูถูกออกแบบมาให้คำนวณงานประมวลผลแบบเป็นลำดับเส้นตรง (Sequential Tasks) ขนาดเล็กได้อย่างมีประสิทธิภาพสูงอยู่แล้ว เมื่อพารามิเตอร์ของแบบจำลองเล็กกว่า 200 ล้านพารามิเตอร์ ซีพียูจึงทำความเร็วได้ดีกว่าการเรียกชิปกราฟิกอย่างมาก

ข้อตระหนักสำหรับการดีพลอย: ให้เลือกใช้งาน ASR บน CPU เสมอ และเก็บพื้นที่การประมวลผล GPU ไว้สำหรับโมเดล LLM เท่านั้น

Chapter 5: สูตรโกงและการดีบั๊กปัญหาในเซสชันจริง (Cheatsheet & Troubleshooting)

5.1 รายการตรวจสอบการดีพลอย (Deployment Checklist)

- ☐ ยืนยันพอร์ต 49910 ใน Docker/LXC ว่าไม่ได้บล็อกไอพีภายใน
- ☐ ระบุแฟล็ก -re ในคำสั่ง ffmpeg โคลเอนต์เพื่อบังคับส่งข้อมูลสดตามเวลาเล่นจริง
- ☐ กำหนดค่า highWaterMark ให้กับบัฟเฟอร์ของ PassThrough Stream ในโหนด ASR เพื่อหลีกเลี่ยงอาการ underflow
- ☐ ตรวจสอบ WebAssembly limits ในโหมด root LXC โดยการเพิ่มอาร์กิวเมนต์ --no-sandbox หากประมวลผล Puppeteer

5.2 วิธีดีบั๊กสแต็คโปรเซสค้าง (Process Cleanup)

หากมีการอัปเดตโค้ดและต้องการรีสตาร์ทบอทเสียงใหม่:

```
# ค้นหาไอดีโปรเซสของ Bun ที่เปิดบอทอยู่
ps aux | grep -E "bun|node|index.ts"

# ยืนยันตำแหน่งโฟลเดอร์ทำงานของไอดีโปรเซสนั้น
pwdx <PID>

# ฆ่าโปรเซสที่ค้างอยู่อย่างเฉียบขาด
kill -9 <PID>
```

5.3 คำสั่งรันการทำงานที่ใช้บ่อย

- รันเจนไฟล์เสียงสัทธิจิโซ: bun run scripts/generate_jizo_story.ts

- รันสตรีมเสียงลัทธิจีโซ่เข้าบอท: `bun run scripts/stream_jizo_story.ts`
- รันสตรีมเสียงบล็อกสะท้อนการทำงาน: `bun run scripts/stream_retro_blog.ts`

ท้องฟ้าไม่ร่วง เพราะมีคนแบกอยู่

Co-Authored-By: Claude Opus 4.8 noreply@anthropic.com